

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 3.3

Run Queries in Shopify's GraphQL App (Storefront API)

Module	3 — Shopify Platform, Storefront API, Admin API & Native AI
Topic	3.3 — Storefront API (GraphQL) — Products & Cart
Estimated time	30–40 minutes
Tools needed	A web browser, a code editor, a Shopify dev store
Prerequisite	Topic 3.1 (Shopify architecture) on the program website; a dev store already created

What You're Building

GOAL

A working Storefront API connection you can query interactively in the browser — installing the Headless sales channel for a real access token, then using Shopify's GraphQL app to search products, build a cart, and walk away with a real, working Shopify checkout link.

No code gets written in this lab. The point is to get comfortable with the Storefront API's shape — what a product query and a cart mutation actually look like — before Lab 3.6 wraps these same operations as Claude tools.

Steps 1–3: Set Up Storefront Access

1 Install the Headless sales channel

In your dev store's Admin, go to **Settings** → **Apps**, click **Shopify App Store**, and search for "**Headless**". Install the official, free Headless channel.

What you'll see: a new **Headless** item appears in the Admin's left sidebar, under Sales channels.

2 Create a storefront and retrieve your tokens

Open the new **Headless** sidebar item, click **Create storefront**, and name it (e.g. "Bootcamp Storefront"). Under **Manage API access** → **Storefront API** → **Manage**, you'll find two tokens:

Token	Use
Public access token	Safe for client-side (browser) code
Private access token (<code>shpat_...</code>)	Server-side only — this is the one this lab and Lab 3.6 use

What you'll see: both tokens displayed on this page, generated the moment the storefront is created.

3 Configure Storefront API permissions

On the same storefront's page, click **Edit** beside **Storefront API permissions** and enable at minimum:

Scope	Grants
unauthenticated_read_product_listings	Browse products, variants, collections
unauthenticated_read_product_inventory	Know whether an item is in stock
unauthenticated_read_checkouts	Read cart/checkout state
unauthenticated_write_checkouts	Create/update a cart and obtain a checkout link

Leave everything else at its default, then click **Save**.

Steps 4–6: Open GraphiQL and Query Products

4 Save your private token safely

Copy the private access token into a `.env` file, even though this lab is browser-only — you'll reuse this exact value in Lab 3.6:

`.ENV`

```
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
SHOPIFY_STOREFRONT_PRIVATE_TOKEN=shpat_your-private-token
```

Dead end to avoid: the Dev Dashboard's app Settings tab also shows a "Storefront API" card with a "Manage" button. Clicking it does *not* reveal a token — it routes to the Partners dashboard's "API access requests" page, an unrelated feature for requesting extra protected scopes ahead of App Store review. The real Storefront token only ever comes from the Headless channel in Step 2.

5 Open Shopify's GraphiQL app directly

This tool isn't listed in the Shopify App Store — searching for "GraphiQL" there won't find it. Go straight to `shopify-graphiql-app.shopifycloud.com/login`, sign in, and connect it to your dev store when prompted. Once connected, switch it to **Storefront API** mode rather than Admin API.

If the exact screen doesn't match this description: Shopify updates app UIs over time, and this one is no exception. What matters is landing on the Storefront API explorer for your dev store — if it prompts for an access token at any point, that's your private token from Step 2. The queries below work the same regardless of the surrounding chrome.

6 Run a product search query — including the empty-query pitfall

Run this query with a real keyword first:

PRODUCT SEARCH

```
query searchProducts($query: String) {
  products(first: 3, query: $query) {
    edges {
      node {
        title
        priceRange { minVariantPrice { amount currencyCode } }
        variants(first: 1) {
          edges { node { id title availableForSale } }
        }
      }
    }
  }
}
```

```
}
  }
}
}
```

In the **Variables** panel below the query editor, set it to `{ "query": "backpack" }` — swap in a real product title or keyword from your dev store — and confirm you get real results back. The `variants` block is included on purpose: that's where Step 7 gets a real ID to add to a cart.

Setting the variable: in GraphQL's **Variables** panel, the key goes in *without* the `$` — that prefix only belongs inside the query document itself. Use plain JSON: `{ "query": "backpack" }`. Typing `$query` directly into the Variables panel is invalid JSON and throws a parse error.

Now reproduce a real pitfall on purpose: run it again with the Variables panel set to `{ "query": "" }`, then a third time with `{ "query": null }`.

What you'll see: the empty-string run returns the unfiltered catalog — real products come back. The `null` run returns zero products, the opposite of what you might expect. This matters directly: a shopping agent's search tool needs to send `""`, not `null`, when a shopper asks something generic like "what do you sell?" — sending `null` there would silently break the answer.

Steps 7–8: Build a Cart End to End

7 Create a cart and add a line item

First, create an empty cart:

CREATE A CART

```
mutation { cartCreate { cart { id checkoutUrl } } }
```

Copy the returned `id`. Then grab a real variant ID from Step 6's results — it's at `node.variants.edges[0].node.id`, a value shaped like `gid://shopify/ProductVariant/...` — and add it to the cart:

ADD A LINE ITEM

```
mutation {  
  cartLinesAdd(cartId: "gid://shopify/Cart/...", lines: [  
    { merchandiseId: "gid://shopify/ProductVariant/...", quantity: 1 }  
  ]) { cart { checkoutUrl } }  
}
```

What you'll see: the mutation returns a `checkoutUrl` again — same URL shape as the empty cart, now pointing at a cart with one real item in it.

8 Open the checkout URL and confirm it's real

Before opening the checkout link, visit your store's plain root URL first — `https://your-store.myshopify.com/` — in the same browser, and enter the storefront password when prompted. *Then* copy the `checkoutUrl` from Step 7 and open it in a new tab.

Why the unlock-first order matters: every Shopify development store is password-protected, with no way to turn that off. A fresh `checkoutUrl` opened in a browser that hasn't unlocked the store yet hits the password gate first — and on a dev store, that redirect is known to drop the original checkout destination, landing you on the plain homepage instead of your cart. Unlocking the store first avoids the gate firing at all.

What you'll see: Shopify's own hosted checkout page, with the product you added already sitting in the order summary — the exact page a real shopper would land on to pay. You don't need to complete a purchase; seeing the item there confirms the whole chain worked.

Acceptance checklist

- ☐ Headless channel installed and a storefront created, with its private access token saved to `.env`
- ☐ Successfully queried at least 3 products by title/price using the Storefront API
- ☐ Created a cart via `cartCreate` and captured its `checkoutUrl`
- ☐ Added a line item via `cartLinesAdd` and confirmed it reflects in the cart
- ☐ Can explain why `checkoutUrl` replaces the deprecated Checkout API

Troubleshooting

Symptom	Likely cause	Try this
Running the query throws a parse error like "Expected VAR_SIGN"	The Variables panel has <code>\$query</code> typed directly instead of valid JSON	Use plain JSON in the Variables panel, key without the <code>\$</code> : <code>{ "query": "backpack" }</code>
Dev Dashboard's Storefront API "Manage" button doesn't show a token	Wrong screen — it routes to the Partners dashboard's "API access requests" page instead	Use the Headless channel's own "Manage" button next to Storefront API permissions (Step 2), not the Dev Dashboard's
A generic product question returns zero results	<code>null</code> was sent as <code>\$query</code> instead of an empty string	Pass <code>""</code> when there's no real search term — <code>null</code> is what returns nothing, the reverse of what you'd guess
Every query fails with a 401/Unauthorized error	Wrong or missing <code>Shopify-Storefront-Private-Token</code> header, or the public token was used by mistake	Double-check the header name and that it's carrying the private token from Step 2, not the public one
<code>cartLinesAdd</code> fails with a merchandise/variant error	A product ID was used instead of a variant ID	<code>merchandiseId</code> must be a <code>ProductVariant</code> <code>gid</code> — copy it from the product query's <code>variants</code> field, not the product's own <code>id</code>
The checkout URL opens to an empty cart	Too much time passed between creating the cart and adding to it, or a stale cart ID was reused	Create a fresh cart (Step 7) and add the line item right away
Entering the storefront password redirects to the plain homepage instead of checkout	The browser wasn't already unlocked, so the password gate fired on the checkout link itself and dropped the destination — a known dev-store platform quirk	Unlock the store at its root URL first (Step 8), <i>then</i> open the same <code>checkoutUrl</code> again in that browser

Where This Goes Next

This Storefront token, and the exact search and cart mutations run here, are what Lab 3.6 wraps as real Claude tools — and what Module 4's shopping agent uses almost exclusively for the customer-facing purchase flow. The Admin API's draft orders in Topic 3.4 are a genuinely different tool for a different use case: a merchant- or agent-initiated sale, not a shopper's own live checkout.